PRACTICAL- 6

Problem Definition

Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

1. How is JSON fetched, parsed, and rendered?

JSON data is fetched using the **Fetch API**, converted into JavaScript objects, and then displayed dynamically on the webpage.

2. Which array methods are used for search, filter, sort, and pagination?

Methods like **filter()**, **sort()**, and **slice()** are used to search, filter, sort, and display data page by page.

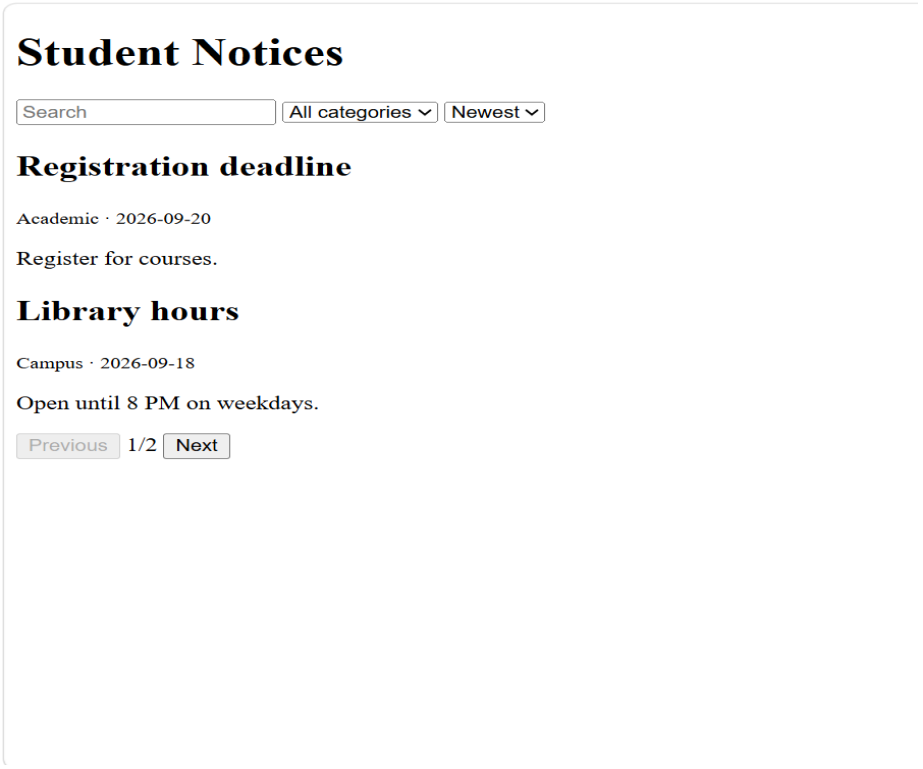
3. How are loading and error states handled?

Loading messages are shown while data is being fetched, and errors are handled using **.catch()** if the JSON cannot be loaded.

4. How is modularity maintained in JavaScript files?

Different functions are used for separate tasks such as fetching, filtering, sorting, rendering, and pagination, making the code easier to manage.

SCREENSHOT:



Student Notices

Search | All categories | Newest

Midterm timetable

Academic · 2026-09-15

Check your examination schedule.

Campus clean-up

Events · 2026-09-12

Volunteer this Saturday.

Previous 2/2 Next

Student Notices

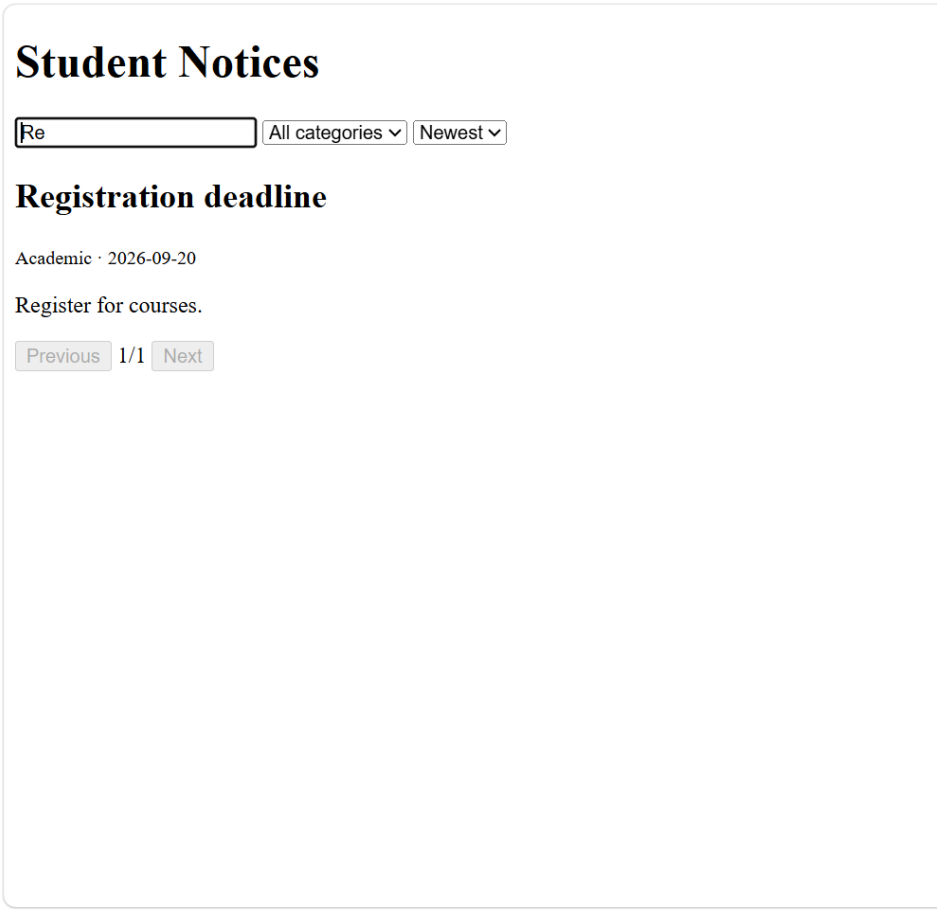
Midt | All categories | Newest

Midterm timetable

Academic · 2026-09-15

Check your examination schedule.

Previous 1/1 Next



Implementation

- Created an external **JSON file** containing student/event data.
- Used the **Fetch API** to load data from the JSON file.
- Parsed the JSON response and converted it into JavaScript data.
- Dynamically displayed the JSON records on the webpage.
- Implemented **search functionality** to find specific records.
- Added **filter functionality** based on available data.
- Implemented **sorting** to arrange the displayed records.
- Added **pagination** to display records page by page.
- Used JavaScript array methods such as filter(), sort(), and slice().
- Added error handling when the JSON file cannot be loaded.

- Implemented password strength checking.
- Used proper labels and input types for accessibility.

Challenges

- Fetching and correctly parsing data from the external JSON file.
- Handling errors when the JSON file is missing or cannot be loaded.
- Implementing search and filter logic correctly.

- Sorting the records without affecting the original data unnecessarily.
- Managing pagination and displaying the correct records on each page.
- Dynamically updating the webpage using JavaScript.
- Handling loading and error states properly.

GitHub Repository

https://github.com/25dce037-tech/Student-Hub-Portal.git

Conclusion :

The external JSON data was successfully fetched using the **Fetch API** and dynamically displayed on the webpage. Search, filtering, sorting, and pagination were implemented using JavaScript array methods, making the application more dynamic and user-friendly.